

TypeScript for This Project

An introduction to TypeScript from first principles, followed by every syntax pattern this codebase actually uses — with real examples pulled directly from the source, not generic textbook samples.

You already know JavaScript. TypeScript adds exactly one new idea on top of it — types — and this document only covers the subset of that idea this project actually reaches for. A search across `renderer/src` found 82 `interface` declarations, 24 `type` aliases, zero `enum`s, and exactly one generic function — that shape (lots of object/union types, almost no advanced generics) is typical of a UI codebase, and it's exactly what this guide focuses on.

Contents

1. What TypeScript is, and isn't — `compiles away`, `checks only`
2. Basic type annotations — `variables`, `function params/returns`
3. `interface` vs `type` — which this project uses, and when
4. Union types — the workhorse of this codebase — `string literal unions` instead of `enums`
5. Optional properties vs. nullable fields — `?:` vs `| null`
6. Arrays, tuples, and `Record<K, V>` — `typed lookup tables`
7. The one generic function in this project — `apiClient.ts`'s `request<T>`
8. `as const` — locking down literal types — why the wizard's `step lookup` needs it
9. Typing React components and props — a `real component`, annotated
10. Typing a Zustand store, start to finish — `shellStore.ts`, fully walked through
11. `import type` and strictness settings — what `tsconfig.base.json` turns on
12. Glossary — quick reference

1. What TypeScript is, and isn't

TypeScript is JavaScript with one thing added: before your code runs, a separate checking step reads every file and verifies that values are used consistently with the "shape" you declared for them. If a function says it wants a `number` and you hand it a string, the checker flags it immediately — in your editor, red squiggly line and all — instead of the mistake surfacing later as a crash or, worse, a silent wrong answer while the app is running.

Two things TypeScript is **not**:

NOT A NEW RUNTIME

Every `.ts` / `.tsx` file is compiled down to plain JavaScript before it ever runs. The type annotations are **deleted entirely** in that step — a browser or Node.js never sees the word `string` anywhere. Nothing about *runtime behavior* changes; only what gets checked *before* runtime does.

NOT A VALIDATION LIBRARY

TypeScript can't check data that arrives while the app is running — like a JSON response from the Python backend. If the backend sent the wrong shape, TypeScript would not catch it; it only checks that *your own code* is internally consistent with the shapes you declared.

2. Basic type annotations

The annotation always comes after a colon, on variables, function parameters, and function return values:

```
let count: number = 0;
let label: string = 'reticent';
let done: boolean = false;

function fmtClock(totalSeconds: number): string { // param type, then return type
  const m = Math.floor(totalSeconds / 60);
  return `${m}`;
}
```

Most of the time in this codebase, you won't see a variable's type written out like `let count: number` at all — TypeScript **infers** it automatically from the value on the right (`let count = 0` is already known to be a `number`). Explicit annotations show up mainly on *function parameters* (which have no initial value to infer from) and on empty containers.

3. `interface` VS `type`

Both describe the shape of an object. This project's convention, visible across every file, is: **`interface` for the shape of an object** (especially anything mirroring a backend API payload), and **`type` for everything else** — unions, function signatures, and simple aliases.

```
// renderer/src/types/api.ts – interface: the shape of a backend response
export interface UserOut {
  id: string;
  display_name: string;
  native_language: string;
  target_language: string;
  cefr_level: string | null; // see Section 5 – nullable, not optional
  created_at: string;
  onboarding_completed_at: string | null;
}

// same file – type: a fixed set of allowed string values, not an object shape
export type LlmMode = 'local' | 'api';
```

Field names in these interfaces (`display_name` , not `displayName`) deliberately match the Python backend's JSON exactly — this interface exists specifically to describe "whatever shape a fetch response from `/users/:id` has," so it uses the backend's own naming, not this project's usual camelCase JS convention.

4. Union types — the workhorse of this codebase

A union type, written with `|` , means "exactly one of these specific values, and nothing else." This is by far the most common custom type in the project — `ScreenKey` (which screen is currently showing) is a representative example, from

`renderer/src/features/shell/navConfig.ts` :

```
export type ScreenKey =  
  | 'dashboard' | 'library' | 'player' | 'bookshelf' | 'reader'  
  | 'vocab' | 'word' | 'review' | 'conv' | 'convlive'  
  | 'report' | 'challenge' | 'forest' | 'settings';
```

Anywhere a value is typed `ScreenKey` , TypeScript rejects anything that isn't one of those exact strings — `goScreen('dashbord')` (typo) is a compile error, not a runtime bug discovered by clicking around. This is also the entire reason the sidebar-navigation switch statement in `App.tsx` is safe to write as a plain `switch` : TypeScript knows the full, closed list of possible values `screen` can ever hold.

Why not a JavaScript/TypeScript `enum` instead? An `enum` would work too, but it compiles to a small real JavaScript object that exists at runtime — a union of string literals compiles to *nothing at all* (fully erased, per Section 1), and it already matches plain JSON strings coming back from the Python backend without any conversion step. This project uses zero `enum` s anywhere — string literal unions cover every case that came up.

5. Optional properties vs. nullable fields

These look similar but answer different questions, and this project is deliberate about which one it uses where:

OPTIONAL — SUB?: STRING

"The caller *may choose not to pass* this property at all." Used for component props that have a sensible default when omitted.

```
interface NavItem {
  key: ScreenKey;
  label: string;
  badge?: string; // Review has
one, most don't
}
```

NULLABLE — CEFR_LEVEL: STRING | NULL

"This property is *always present*, but its value might genuinely be the absence of data." Used for backend fields the database can hold as SQL `NULL`.

```
interface UserOut {
  // always there, but null until
  // placement is completed
  ceفر_level: string | null;
}
```

Mixing these up would be a real, meaningful bug: writing `badge?: string` for a field the backend always sends as either a string or `null` would let `undefined` slip through unnoticed; writing `ceفر_level: string | null` for an optional UI prop would force every caller to explicitly pass `null` even when they have nothing to say about it.

6. Arrays, tuples, and `Record<K, V>`

Plain arrays are written `string[]` or, equivalently, `Array<string>` — both appear in this codebase. Two more specific shapes come up often enough to call out:

6.1 Tuples — a fixed-length array where each slot has its own type

```
// renderer/src/features/shell/navConfig.ts
export const SCREEN_TITLES: Record<ScreenKey, [string, string]> = {
  dashboard: ['Dashboard', 'What is due, how you are trending, what to do next'],
  review:    ['Review', 'FSRS v4 · target retention 0.90 · interleaved queue'],
  // ...
};
```

`[string, string]` is a **tuple**: exactly two elements, both strings — not "an array of strings of any length." `ScreenHeader.tsx` deconstructs it as `const [title, staticSub] = SCREEN_TITLES[screen]`, and TypeScript knows `title` and `staticSub` are both definitely strings, not "maybe undefined if the array happened to be shorter."

6.2 `Record<K, V>` — a typed lookup table

The same line above is also a `Record` — TypeScript's built-in type for "an object used as a dictionary, where every key is of type `K` and every value is of type `V`." `Record<ScreenKey, [string, string]>` means: this object **must** have exactly one entry for every single value in the `ScreenKey` union, no more, no fewer.

Why this matters in practice: this is the exact mechanism that caught the current project honest — when a new screen (say, `'forest'`) is added to `ScreenKey` but its title/subtitle entry is forgotten in `SCREEN_TITLES`, TypeScript refuses to compile until every key is filled in. This project's `Settings.tsx` uses the same pattern for `SETTINGS_GROUPS: Record<SettingsGroupName, SettingGroup>`.

7. The one generic function in this project

A **generic** lets a function or type work with "whatever type you tell it to, this one time" instead of one hardcoded type. This codebase has exactly one, and it's worth understanding well because every single API call in the app funnels through it — `renderer/src/lib/apiClient.ts`:

```
async function request<T>(path: string, init?: RequestInit): Promise<T> {
  const res = await fetch(`${baseUrl}${path}`, { /* ... */ });
  return (await res.json()) as T;
}

export const api = {
  get: <T>(path: string) => request<T>(path, { method: 'GET' }),
  // ...
};
```

`<T>` is a placeholder type, filled in at the call site. Calling `api.get<UserOut>('/users/123')` tells TypeScript "trust me, whatever JSON comes back has the shape of `UserOut`" — and from that point on, everywhere that result is used, autocomplete and type-checking treat it as a real `UserOut`, not a vague `any`. This is also, honestly, a small leap of faith: TypeScript cannot verify the server actually sent that shape (Section 1's second point) — the generic only documents and enforces the *assumption*, which is why the interfaces in `types/api.ts` need to be kept accurate by hand as the backend changes.

8. `as const` — locking down literal types

By default, TypeScript "widens" a literal value to its general type — the number `1` inside a plain array becomes typed as `number`, not "specifically 1." Usually that's fine, but this project has one spot where it isn't:

```
// renderer/src/features/onboarding/onboardingConfig.ts
export const ONBOARDING_STEPS = [
  { n: 1, name: 'Profile', title: 'Who is learning', body: '...' },
  { n: 2, name: 'Placement', /* ... */ },
  // ... n: 3, 4, 5
] as const; ← without this, `n` would just be typed `number`
```

```
// renderer/src/features/onboarding/OnboardingWizard.tsx
const STEP_COMPONENTS = { 1: StepProfile, 2: StepPlacement, 3: StepEngine, 4: StepHabit, 5:
StepCompanion };
const StepComponent = STEP_COMPONENTS[step]; // step must be exactly 1|2|3|4|5, not "some
number"
```

`as const` tells TypeScript "treat every value in here as its exact literal, and make the whole structure read-only." Without it, `ONBOARDING_STEPS[0].n` would be a generic `number`, and the `OnboardingStep` type (`1 | 2 | 3 | 4 | 5`, used all over `onboardingStore.ts`) couldn't be derived from it — the lookup in `STEP_COMPONENTS[step]` would no longer be provably safe, since *some number* might not be a valid key.

9. Typing React components and props

A component's props are typed with an object type, usually written inline right where the function is defined. From `renderer/src/features/onboarding/steps/StepProfile.tsx`:

```
function TextField({
  label, sub, value, onChange, placeholder,
}): {
  label: string;
  sub: string;
  value: string;
  onChange: (v: string) => void; // a function type: takes a string, returns nothing
  placeholder?: string; // optional – Section 5
}) {
  /* ... */
}
```

`(v: string) => void` is a **function type** — it describes the shape of a callback, not a specific function. Anyone using `<TextField onChange={...}>` must pass a function that accepts one string and doesn't need to return anything; passing a function that returns a number is still fine (the return value is simply unused), but passing one that expects two arguments would be a compile error.

10. Typing a Zustand store, start to finish

Since you have `shellStore.ts` open, here is exactly how it's typed, piece by piece — this same shape repeats in `appStore.ts` and `onboardingStore.ts`.

```
interface ShellState {
  screen: ScreenKey;           // — data fields —
  collapsed: boolean;
  theme: 'dark' | 'light';     // an inline union — didn't need its own named type
  convScenario: string;

  goScreen: (key: ScreenKey) => void; // — actions — (functions that change the data)
  goConvLive: (scenario: string) => void;
  toggleTheme: () => void;
}

export const useShellStore = create<ShellState>((set, get) => ({
  screen: 'dashboard',
  collapsed: false,
  theme: 'dark',
  convScenario: 'Free talk',

  goScreen: (key) => set({ screen: key }), // `key` is inferred as ScreenKey — no re-
  // annotation needed
  goConvLive: (scenario) => set({ screen: 'convlive', collapsed: true, convScenario: scenario
}),
  toggleTheme: () => {
    const next = get().theme === 'dark' ? 'light' : 'dark';
    set({ theme: next });
  },
}));
```

- 1 `interface ShellState` declares the **entire** shape of the store up front — every field and every action, with no implementation yet, just their types.
- 2 `create<ShellState>(...)` is Zustand's own generic function (same idea as Section 7's `request<T>`) — it tells TypeScript "whatever object I return from this initializer function must match `ShellState` exactly."
- 3 Inside, every field/action is checked against that interface. Forget to implement `toggleTheme`, or implement it with the wrong parameter types, and the whole file fails to compile — before you ever click the theme toggle button and find out the hard way.
- 4 Everywhere else in the app, `useShellStore((s) => s.theme)` infers `s` as `ShellState` automatically, so `s.theme` is known to be `'dark' | 'light'` — and `s.thme` (typo) is a compile error, not a silent `undefined`.

11. `import type` and this project's strictness settings

11.1 `import type`

```
import type { ModelTier } from '@types/api'; // used 12 times across renderer/src
```

A plain `import` can pull in both runtime values (functions, components) and types. Writing `import type` tells the compiler "everything imported on this line exists only for type-checking — delete it completely from the compiled JavaScript." This matters because

`tsconfig.base.json` sets `isolatedModules: true`, which requires each file to be compilable on its own without seeing other files first; an ordinary import of a type-only name can be ambiguous under that constraint, while `import type` removes any doubt for the build tool (Vite, from the frontend document) about what needs to survive into real JS.

11.2 The strictness flags this project turned on

From `tsconfig.base.json`, shared by both `renderer/` and `electron/`:

SETTING	WHAT IT CATCHES
<code>strict: true</code>	Turns on the full bundle of strict checks — no variable is silently <code>any</code> , <code>null</code> / <code>undefined</code> must be handled explicitly, and more. This single flag is why patterns like Section 5's <code>string null</code> have to be written honestly instead of ignored.
<code>noUnusedLocals</code> / <code>noUnusedParameters</code>	A variable or function parameter that's declared but never used is a compile error, not just an editor warning — keeps dead leftovers from previous edits from quietly accumulating.
<code>noFallthroughCasesInSwitch</code>	In a <code>switch</code> statement (like <code>App.tsx</code> 's screen router from the frontend document), forgetting a <code>return</code> / <code>break</code> and accidentally "falling through" into the next case is a compile error.

12. Glossary — quick reference

TERM	ONE-LINE DEFINITION
type annotation	The <code>: Type</code> written after a variable/parameter name.
type inference	TypeScript figuring out a type on its own from context, without an explicit annotation.
interface	Describes the shape of an object — this project's convention for anything mirroring backend data.
type alias	A name for any type — this project's convention for unions and function-shape types.
union type (<code> </code>)	"Exactly one of these listed possibilities" — this codebase's replacement for <code>enum</code> .
optional property (<code>?:</code>)	The caller may omit this field entirely.
nullable (<code> null</code>)	The field is always present, but its value may be null.
tuple	A fixed-length array where each position has its own specific type, e.g. <code>[string, string]</code> .
<code>Record<K, V></code>	An object type used as a dictionary — every key of type K maps to a value of type V.
generic (<code><T></code>)	A placeholder type filled in by whoever calls the function, so one function body works for many types.
<code>as const</code>	Locks a literal value/array/object to its exact type instead of letting TypeScript widen it (e.g. keeps <code>1</code> as <code>1</code> , not <code>number</code>).
import type	An import that exists only for type-checking and is fully erased from the compiled JavaScript.
strict mode	The bundle of stricter checks this project enables in <code>tsconfig.base.json</code> , including mandatory null-handling.

Companion reading: *FluencyOS — Frontend, Electron & Onboarding Explained* and *Tailwind CSS for This Project* cover the rest of the frontend stack the same way this document covers TypeScript. Every example above is taken directly from the current `renderer/src` source, with file paths given so each one can be opened and shown live.